Direct and Fictitious Self-Play: An Analysis of MARL With Dots and Boxes

Cullen McCaleb and Edward Kiboma

Abstract

The classic game dots and boxes is a good example of a two-player zero-sum game, where one player's gain is equivalent to the other player's loss. Two-player zero sum games fall under the umbrella of multi-agent reinforcement learning (MARL) problems within reinforcement learning (RL). Furthermore, self-play is a powerful method of training in MARL where an agent learns by repeatedly playing itself, and tends to work well on zero-sum games. In fact, many variations of self-play have found success in these types of games, and this report will analyze two of them: direct self-play and fictitious self-play. Our goal is to use dots and boxes as an environment to evaluate these training methods, determine their strengths and weaknesses, and ascertain which method is better suited for this game.

1 Introduction

One of the emerging topics in Reinforcement Learning right now is the concept of multi-agent reinforcement learning, or MARL. MARL focuses on studying the behavior of multiple agents that interact with each other in the same environment. These agents can either work together towards the same goals, or have rewards that oppose each other. The primary problem with MARL, however, is that the environment seems non-stationary to each agent resulting in challenges with coordination and communication. Ultimately, it is difficult to achieve convergence using MARL.

Self-play is a technique in MARL where the agents are neither cooperative nor competitive, but identical. Each agent is derived from its ancestors to compete with past versions of itself. Direct self-play (DSP) is the traditional method, where each agents competes with the most recent version of itself. Some concerns with this strategy are that it will result in cycling behaviors, overfitting, and in general forgetting some of what it learns from past versions. The solution to these problems is a variation of traditional self-play, where the agent plays against a set of strategies from previous iterations. This way, different strategies are compared with each other so that more robust and adaptive strategies are developed. This technique is referred to as either fictitious self-play (FSP)

or fictitious self-play against frozen agents. While these strategies are similar, the former focuses more on playing against an average of past behaviors that also learn, while the ladder typically uses the best performing previous copy of itself for training. This study will use a hybrid between the two as the implementation is slightly more convenient, and makes sense for our goals. The motivation behind observing the benefits of FSP is to determine if these methods can quickly converge on an optimal strategy to outperform the DSP method.

This problem is important because the impact of these results go past simple strategy games, including robotics, negotiation systems, and autonomous decision-making. By analyzing the differences between DSP and FSP, we can gain insights into building more effective AI for competitive environments, ultimately improving how machines learn and adapt in dynamic, multi-agent settings.

2 Background Related Work

2.1 Direct Self-Play

The inspiration for this paper comes from the AlphaZero reinforcement learning model which is an algorithm that "achieved superhuman performance in the game of Go" and was applied to Chess and Shogi (Silver et al., 2017). This algorithm utilizes direct self-play, as it continually plays the most recent version of itself. Monte Carlo tree search is used to guide the training process, and eventually update a neural network based on the games against its previous version. AlphaZero matches or outperforms the top engines in Chess, Go, and Shogi, displaying the power of direct self-play. [2]

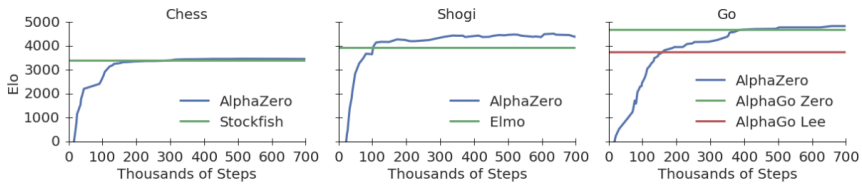


Figure 1: AlphaZero consistently performing as good or better than other top engines.

2.2 Fictitious Self-Play

Self-play with past opponents will ideally mitigate some of the shortcomings of direct self-play by ensuring the agent retains knowledge of past strategies. A great example of fictitious self-play with frozen agents, and relevant to the previous example in section 2.1, is the algorithm AlphaGo Zero, which AlphaZero

is based on. AlphaGo Zero is one of the best performing Go engines, and generated self-play games "by the best player from all previous iterations." This best-player is frozen and the aglorithm does not update its Q-table. [2]

On the other hand, DeepMind's AlphaStar (Vinyals et al., 2019), which managed to master the complex game StarCraft II, uses regular fictitious self-play. It maintained a population of historical versions of itself to ensure robustness against a wide range of playstyles. According to this study, the goal of fictitious self-play is to "avoid cycles by computing a best response against a uniform mixture of all previous policies; the mixture converges to a Nash equilibrium in two-player zero-sum games." Thus, this technique should resolve some convergence problems that direct self-play is hypothesized to face.[4]

Additionally, an AI called Pluribus (fictitious), which can beat elite human professionals at Texas Hold'em Poker, utilizes fictitious self-play as well (Brown and Sandholm, 2019). Pluribus also approximates a Nash equilibrium strategy, or "a list of strategies, one for each player, in which no player can improve by deviating to a different strategy." This study conveniently describes the idea that to "solve" a two-player zero-sum game means to find an exact Nash equilibrium. Thus, this is the goal of both fictitious self-play and direct self-play. [1]

3 Experimental Design

3.1 Environment - Markov Decision Process (MDP)

This problem will be framed as an MDP, with the state-action space being the board configuration. For dots and boxes, this includes the lines that have been drawn, and which boxes belong to each player. The action space will be each empty edge, where a line could be drawn, and the reward function will give a small reward for completing a box, a small negative reward if the opponent completes a box, and a huge positive or negative reward for a win or a loss. Although the code does not explicitly formalize these MDP components, the interaction between the agent and environment follows the MDP framework, and allows Q-learning to be applied successfully.

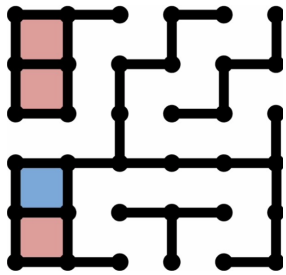


Figure 2: Visual of the dots and boxes state space and action space

- The grid is represented using two separate 2D Numpy arrays. One is 3x4 to represent the possible horizontal lines that can be drawn, and the other is 4x3 to represent the possible vertical lines that can be drawn. Together, they represent a 4x4 grid of dots where a total of nine boxes can be drawn.
- The action space is represented as a tuple, where ("h", x, y) means draw a horizontal line and ("v", x, y) is for a vertical line. At the beginning there are 24 possible actions, and after each line is drawn that action is removed as a possible action.
- The agent receives a +1 reward if its action completes a box, a +10 reward for winning the game, and a -10 reward for losing.
- Each episode contains one game and is completed once all nine boxes are drawn. After the episode, the winner is determined and the agent's Q-table is updated. There are a total of 10,000 episodes.

3.2 Q-Learning

Q-learning is the primary reinforcement learning algorithm used to train the agent in both DSP and the FSP hybrid. The agent incrementally updates a Q-table to estimate the expected return for each state-action pair, using the following update rule from Sutton and Barto [3]:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

Here, α is the learning rate, γ is the discount factor, and R_{t+1} is the immediate reward after taking action A_t in state S_t. In the experiment, α is set to 0.5 and γ to 0.9.

The difference in the implementation of Q-learning between the two methods lies in the Q-table. In DSP, the agent updates its Q-table throughout training by playing games against itself. Both players share the same policy, and the Q-values evolve over time as the agent repeatedly plays full games and refines its estimates. For FSP, the agent plays against frozen copies of its past selves,

simulating a non-stationary opponent distribution. A buffer of past agent versions is maintained, and at the start of each episode the opponent is selected randomly from this population.

The action selection strategy for Q-learning will be ϵ-greedy, so that with ϵ probability the agent will choose a random move. Otherwise, it will choose the move with the highest Q-value. The ϵ value used is 0.1.

3.3 Self-Play Implementation

Two modes of self-play training were implemented: Direct Self-Play (DSP) and a hybrid Fictitious Self-Play (FSP) method.

DSP involves the agent playing against itself, with both players using the same Q-table and parameters. During each turn, the agent selects an action using its ϵ-greedy policy and updates the Q-value table based on the observed reward and next state. Since the Q-table is shared across both players, the agent continually learns from both perspectives within a single episode. This setup is simple but can lead to instability as the agent is constantly adapting to its own changing behavior.

FSP is a hybrid between fictitious self-play and fictitious self-play with a frozen agent because we are not choosing the best past agent. Instead, the idea of selecting random past agents is taken from fictitious self-play but we are not updating the Q-table for this random past-agent - it is frozen. We introduce a population of frozen agents, representing past versions of the learning agent. At the start of each episode, the opponent is selected uniformly at random from this pool. These past agents do not learn during the episode; only the current agent's Q-table is updated. This introduces stability into the opponent's policy and exposes the agent to a distribution of past strategies, promoting robustness. The population of past agents grows periodically by appending a copy of the current agent after evaluation checkpoints.

3.4 Evaluation Metrics

To evaluate the extent of the success during learning for these two self-play methods, both a graph and table were generated as visuals to document results. As a note, player 1 is always the learning agent. In DSP, player 2 is the mirror agent and in FSP player 2 is the opponent.

3.4.1 Graph

The graph plots the win rate of the main self-play agent during training, as well as the win rate against a random opponent to evaluate the agent in training against an opponent that was not itself. By random opponent, we mean an opponent that chooses actions at random, not the randomly chosen past agent

during FSP. Both of these values were plotted against the number of episodes on the x-axis.

3.4.2 Table

The table contains the final self-play win rate against itself, the final win rate against the random opponent, and the total wins of player one and player two (both are self-play agents and player one is the main agent we are training). Additionally the score margin is kept track of throughout the training, to see how much player one beats player 2 by. This offers insight on how close the games are to figure out if the agent is dominating or barely winning.

4 Experimental Results

4.1 DSP Results

4.1.1 Graphs

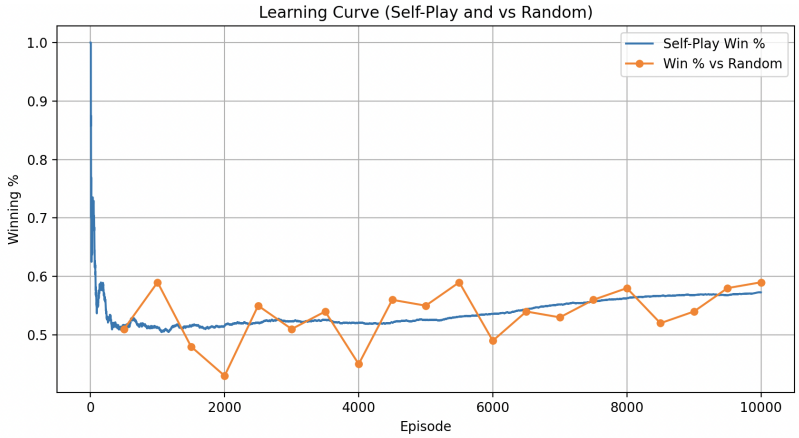
Figure 3: Direct Self-Play Trial 1 Results

Metric	Value
Final Self-Play Win Rate (%)	65.91
Final Win Rate vs Random (%)	53.00
Average Score Margin	1.45
Total Wins (Player 1)	6591
Total Wins (Player 2)	3409

Figure 4: Direct Self-Play Trial 2 Results

Metric	Value
Final Self-Play Win Rate (%)	55.93
Final Win Rate vs Random (%)	63.00
Average Score Margin	0.09
Total Wins (Player 1)	5593
Total Wins (Player 2)	4407

Figure 5: Direct Self-Play Trial 3 Results



Metric	Value
Final Self-Play Win Rate (%)	57.28
Final Win Rate vs Random (%)	59.00
Average Score Margin	0.42
Total Wins (Player 1)	5728
Total Wins (Player 2)	4272

4.1.2 Analysis

The direct self-play results show solid progress in learning. Among all three trials, the self-play win rate for the learning agent steadily increased throughout the trial and ended at an average of 59.7%. Additionally, the final win rate against the random opponent averaged out to be 58.3%, which is well over 50% and shows learning because the trained agent essentially started with random action selection. Finally, the score margin was solid at an average of +0.65 among all three trials.

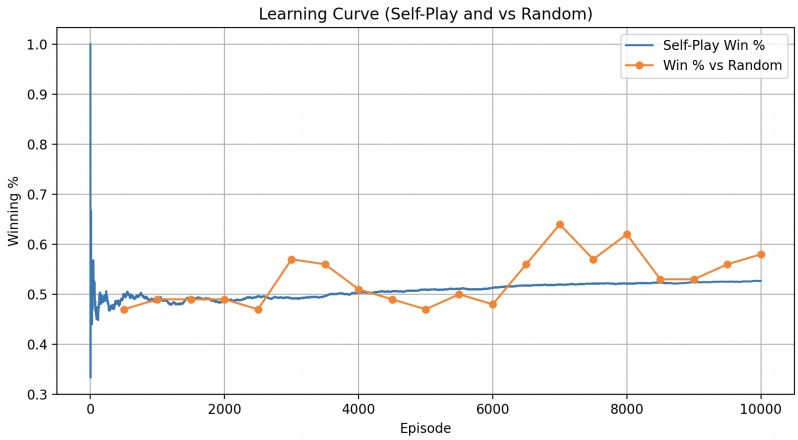
4.2 FSP Results

4.2.1 Graphs

Figure 6: Fictitious Self-Play Trial 1 Results

Metric	Value
Final Self-Play Win Rate (%)	52.60
Final Win Rate vs Random (%)	58.00
Average Score Margin	0.25
Total Wins (Player 1)	5260
Total Wins (Player 2)	4740

Figure 7: Fictitious Self-Play Trial 2 Results



Metric	Value
Final Self-Play Win Rate (%)	52.69
Final Win Rate vs Random (%)	58.00
Average Score Margin	0.30
Total Wins (Player 1)	5269
Total Wins (Player 2)	4731

Figure 8: Fictitious Self-Play Trial 3 Results

Metric	Value
Final Self-Play Win Rate (%)	52.90
Final Win Rate vs Random (%)	57.00
Average Score Margin	0.30
Total Wins (Player 1)	5290
Total Wins (Player 2)	4710

4.2.2 Analysis

The hybrid FSP results show marginal progress in learning, and failed to improve on the DSP results like we hypothesized. The self-play win rate for the learning agent barely increased throughout each trial and in trial 3 there was practically no increase at all. The win rate ended at an average of 52.7%. The final win rate against the random opponent, though, averaged out to be 57.6%, which is again well over 50% and shows a similar amount of learning as the DSP agent. Finally, the score margin was reasonable at an average of +0.28 among all three trials.

4.3 Total Results Analysis

The above results show that the method of DSP performed surprisingly well, averaging better results among the three trials than the hybrid FSP method in self-play win rate, random opponent win-rate, and average score margin. These findings go against our initial hypothesis that FSP would yield better results by exposing the agent to a more stable and diverse distribution of opponents. There are several possible explanations for why DSP performed better.

First of all, although DSP may risk overfitting, the fact that the agent plays against its own current policy means it is always playing against one of the strongest opponents during training, as the most recent policy should be more developed. Also, since FSP chooses random opponents, there is no guarantee that it provides sufficiently diverse or challenging strategies to improve robustness. Additionally, the opponent pool may include weaker agents from early training, which is clearly less optimal than playing the current agent like DSP does. Finally, since FSP does run this risk detailed above, perhaps it just needs more training time due to the complex dynamics it provides.

Despite these differences, both methods achieved win rates significantly above 50% against random opponents, indicating meaningful learning.

5 Conclusion and Future Work

This report explored the application of reinforcement learning to the game of Dots and Boxes using two self-play training regimes: Direct Self-Play and a hybrid form of Fictitious Self-Play. Through Q-learning, we trained agents over 10,000 episodes in both settings, measuring performance based on self-play win rate, win rate against a random opponent, and average score margin. Across three independent trials, DSP consistently outperformed the FSP hybrid in all evaluation metrics.

Although the idea behind FSP is to overcome the shortcomings of DSP like overfitting and behavior cycling, DSP yielded stronger results. The consistency of always training against the most up-to-date version, and therefore probably the smartest version of itself may have provided clearer learning signals than

FSP, which samples from a pool of potentially weaker or outdated opponents. Additionally, while FSP theoretically offers a more stable and diverse training distribution, our hybrid implementation may not have fully captured its benefits.

Future work could explore improvements to the FSP strategy by incorporating weighted opponent sampling, or choosing the best opponents. Additionally, implementing deep Q-networks or even neural networks could enable better scalability, especially for larger board sizes. Algorithms mentioned in the background section use these ideas, and it would be interesting to see if the hybrid idea could work on these networks. Another possible idea is to use our experiment framework but focus more on convergence rather than comparing the strategies based on wins.

References

[1] Noam Brown and Tuomas Sandholm. Superhuman ai for multiplayer poker. *Science*, 365(6456):885–890, 2019.

[2] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.

[3] Richard S Sutton and Andrew G Barto. *Introduction to reinforcement learning*, volume 135. MIT press Cambridge, 1998.

[4] Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.